Development of a turn-based game engine using the Java programming language

Angela Liguori (7112102)

Contents

1 Introduction

The objective of this report is to illustrate the main functionality of the game engine simulator software, underlining the different stages of its development, from the design to its implementation and testing.

1.1 Statement

The software simulates the unfolding of a turn-based video game, its matches consisting of battles where the characters, controlled by players, alternate their chance to choose between attacking or healing.

The game distinguishes between two modes of play: two players, one against the other, or teams consisting of two or more players each, always in equal numbers. This difference can be appreciated only at the user level, as the software is made in such a manner that everything can function either way without special adjustments.

1.2 Architecture

The user interacts with the program via Command-Line interface. The architecture of the system is organised into interconnected packages, in such a way that separates the orchestrating part of the battle from the mechanics of the game's entities. This choice was made to ensure clarity, flexibility and maintainability.

The core of the system is handled by the following packages: `engine`, `observer` and `command`. In particular the `BattleEngine` class manages the coordination of each battle's game cycle, overseeing information regarding turn progression, change in character's statistics and events. This information, stored into `Record` objects to ensure immutability, is used by the `BattleEngine` to update the state of the characters and of the battle itself, with the help of the pattern Observer designed in the `engine` package to notify the changes.

The function of bridge between user's decisions and the engine, is taken over by the `command` package. Due to the use of the Command pattern, each decision gets encapsulated into independent objects containing the actual execution logic. This is how the separation between the user interface and the underlying mechanics of the computational logic is achieved.

The structure of the playable part of the game is assigned to `character`, `skill`, and `state`. The user's point of view begins with characters provided with health points, energy and a set of skills. The condition of the players changes constantly throughout the battle, and is stored into the `CharacterState state` field of the `Character` instance.

Any alteration dynamically affects the efficiency of the characters, modifying the probability of a successful hit or that of a dodge.

A key feature that was kept in mind during the development, is that of the extendability for an eventual Graphic Interface, allowed by following the Open/-Closed Principle, and by keeping responsibilities separated, among other things that will be discussed further in the following sections.

1.3 Employed technologies

The code has been written in the Java programming language, on the Eclipse IDE for Java Developers - 2025-06, and versioned on the GitHub platform, where the repository can be found at the following link:
https://github.com/anjgela/GameEngineSimulator.

The class diagrams and Use Case Diagrams, illustrated in the following paragraphs, were realised using the StarUML software.

The Mock-ups were made using Krita software and Canva.

2 Design

2.1 Use Case Diagram

The designated actors are the users (also referred to throughout this document as the players).

The latter can choose whether to play one versus one or in team mode with a maximum of three players for each team, then each of them gets to choose the type of character to play (`Archer`, `Swordsman`, `Wizard`) and its name. Once every player has picked, the battle begins.

At each turn, the player chooses the preferred `Skill` to use, then the target: an opponent in case of an `AttackSKill`, a member of their own team including oneself if a `HealingSkill` was chosen.

The battle is over once every member of either team has died.

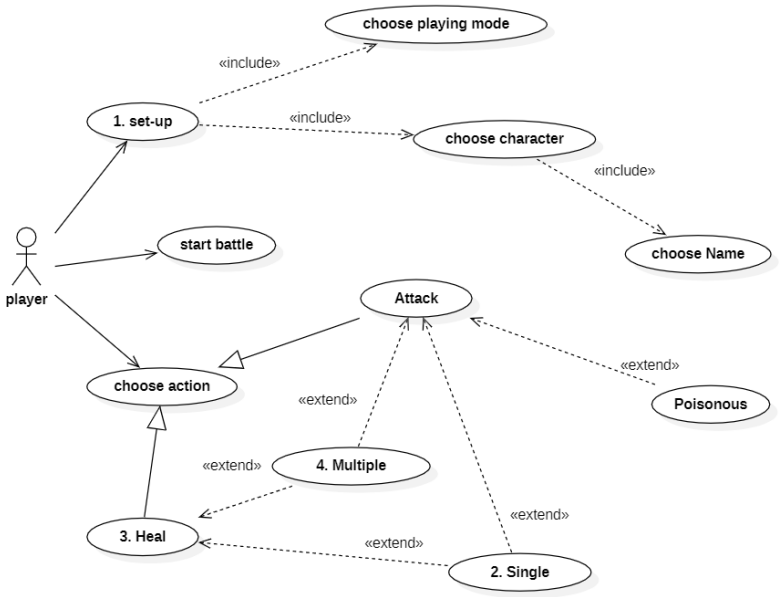

Figure 1: Use Case Diagram Player

2.2 Use Case Templates

The templates displayed below are explanatory of the most significant use cases of this project.

UC-1	**Set-up**
Level	User Goal
Description	The player chooses mode and character.
Actors	User
Ordinary course	1. The user chooses the mode of play. 2. The user picks a character (MK#1). 3. The user picks a name. 4. Control passes to next user, from 2.

Table 1: Set-up

UC-2	**Execute Single Attack**
Level	User Goal
Actors	Player
Description	The player uses an Attack Skill against one single foe.
Ordinary Course	1. The user chooses the "Attack" action. 2. The user selects a "Single" type `Skill` (MK#2). 3. The user chooses the target among members of the opposing team. 4. The corresponding `Command` calculates damage and the target loses health points (MK#3, TST#1).
Alternative Course	4a The target dodges the attack, its health remains unchanged (TST#2). 4b The damage inflicted on the target brings its health below a certain level (20), causing it to transition into the `Slowed` state (TST#3).

Table 2: Single Attack

UC-3	**Healing and Enhancement**
Level	User Goal
Actors	Player
Description	The player heals a teammate, if its health is above a certain level (90), the target enters the Enhanced State.
Ordinary course	1. The player selects action "Heal". 2. The player chooses a target among the members of its own team. 3. The target's health increases, with a maximum value of 100 (TST#4).
Alternative course	3a The target's health after healing reaches a value greater or equal to 90, transitioning to the `Enhanced` state (MK#4, TST#5).

Table 3: Healing

UC-4	**Execute Multiple Attack**
Level	User Goal
Actors	Player
Description	The player uses a Skill `Multiple` to hit a number of targets.
Ordinary Course	1. The player chooses a `Skill` decorated as `Multiple`. 2. The player selects two targets among the members of the opposing team. 3. Both targets take damage (TST#6).
Alternative Course	2a The player is into the `Enhanced` state. The system allows the selection of a maximum of three targets. All three targets take damage.

Table 4: Multiple Attack

UC-5	**Execute Poisonous Attack**
Level	User Goal
Actors	Player
Description	The player uses a Skill `Poisonous`, inflicting damage and altering the target's state.
Ordinary Course	1. The player chooses an `AttackSkill` decorated as `Poisonous`. 2. The player selects a target among the members of the opposing team. 3. The target takes damage and its state transitions to `Poisoned` (TST#7).
Alternative Course	3a The attack is not successful, resulting in the target's health and state remaining unchanged.

Table 5: Poisonous Attack

2.3 Mock-Ups

Here are presented a few mock-ups for a possible graphic interface of the software. They refer to the Use Case Templates in Section 2.2.

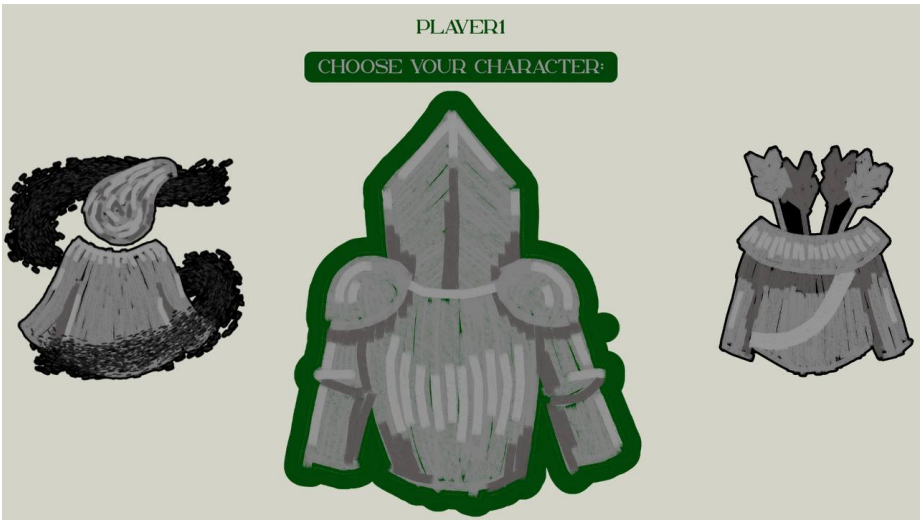

Figure 2: Choose Character - MK#1

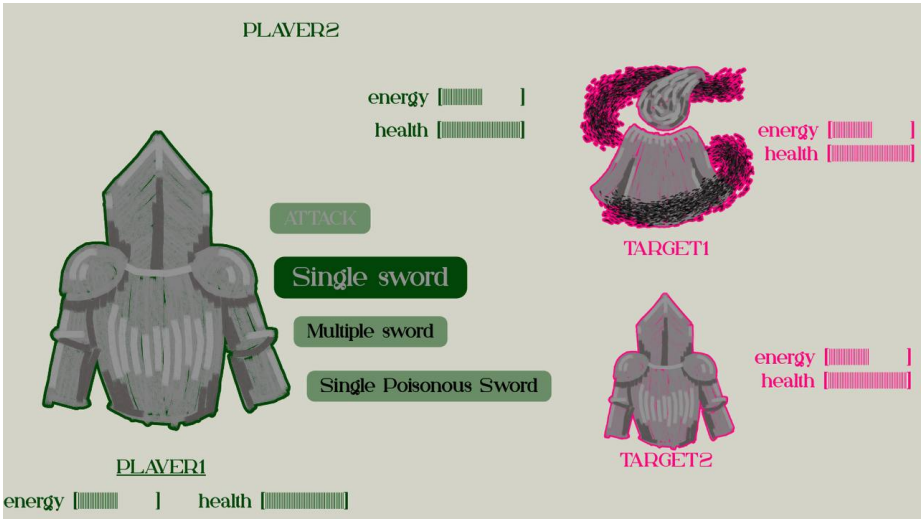

Figure 3: Choose single attack - MK#2

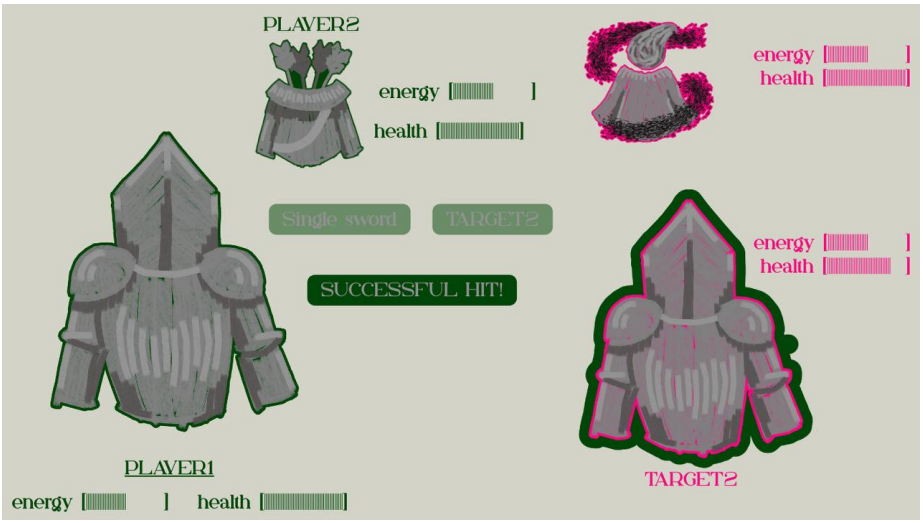

Figure 4: Successful hit - MK#3

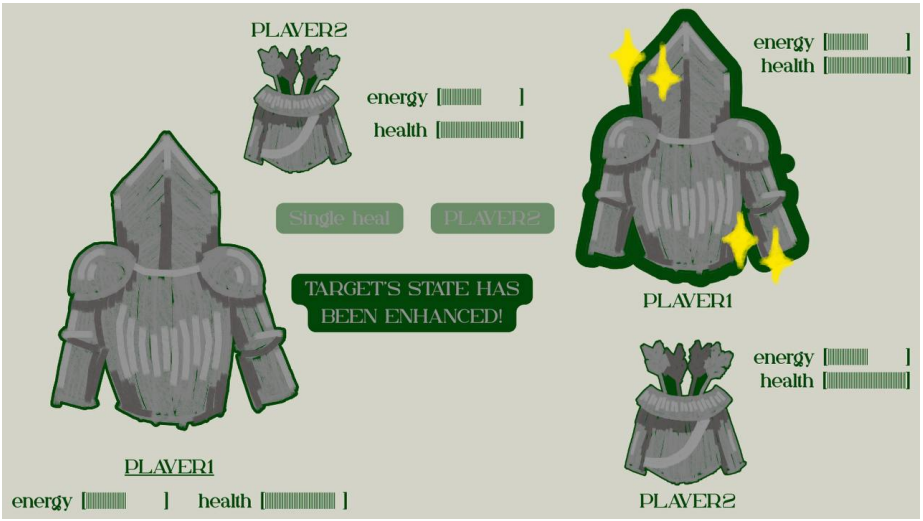

Figure 5: Target's state enhanced - MK#4

2.4 Class Diagrams and packages

This section illustrates the structure of the game engine simulator through detailed class diagrams, organised by their respective packages to reflect the system's architecture.

Figure 6 provides a comprehensive overview of the entire system, highlighting the architectural dependencies and the structural relationships between the various packages. As shown in the diagram, this modular design ensures that the core engine, the playable entities, and the command logic remain decoupled yet interconnected.

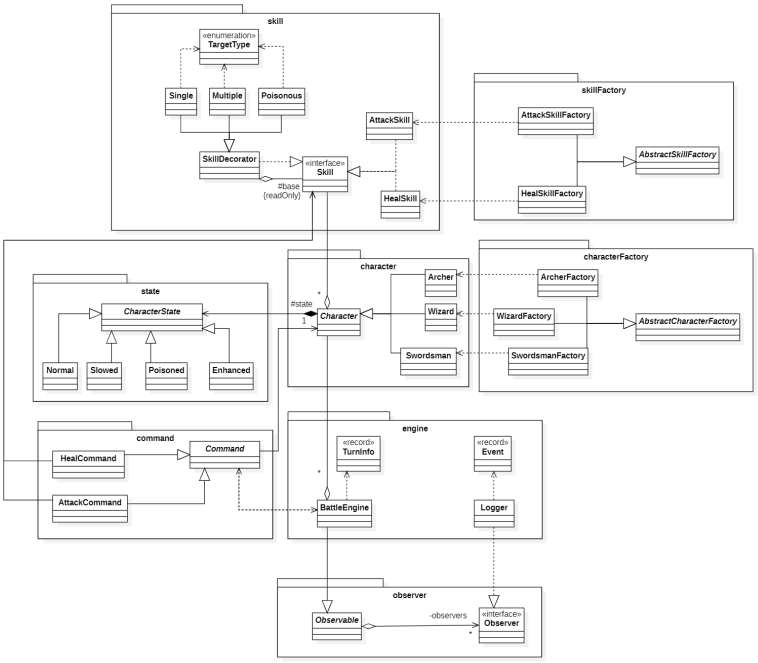

Figure 6: Overall Class Diagram illustrating the architectural dependencies and structural relationships between packages

The different classes are arranged in these packages:

- **engine** : management of every event (actions chosen by users and their consequences) occurring during each turn, and their succession, handling the battles from start to finish.

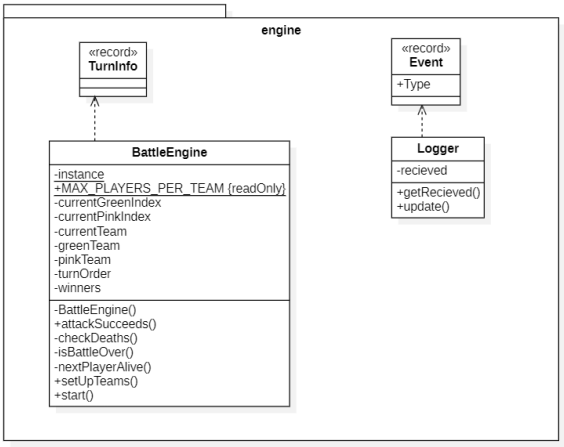

Figure 7: engine package UML

- **observer** : observer design pattern setting the engine structure.

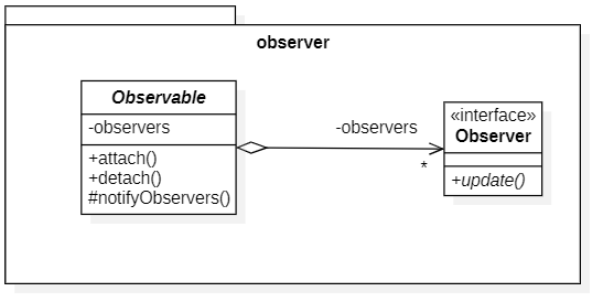

Figure 8: observer package UML

- **command** : formalising and actualising the actions chosen by the players, connecting them to the engine.

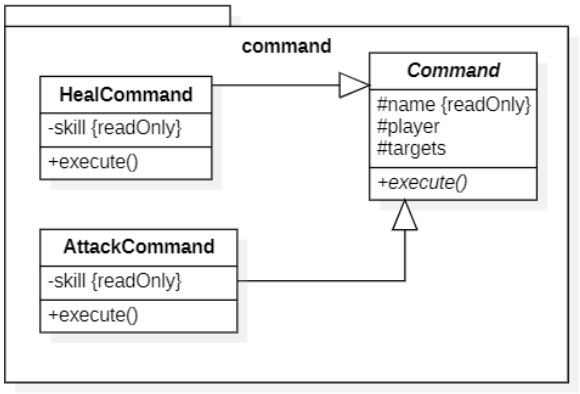

Figure 9: command package UML

- **skill factory** : factory method design pattern used to generate skills associated with the characters. This pattern was chosen with the intent of decoupling creation from usage, for the prospect of extending and better specifying creation behaviour.

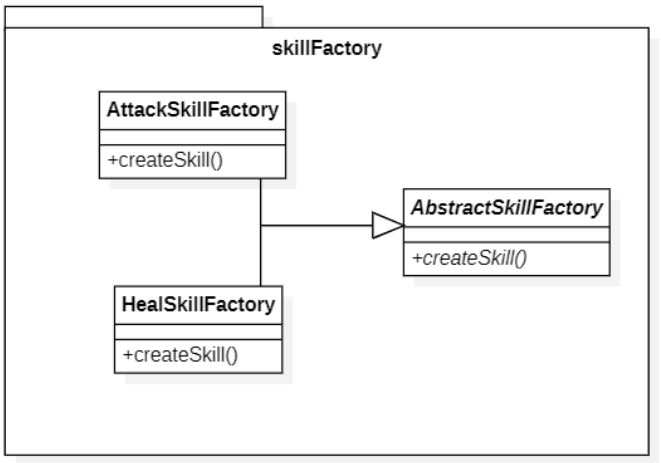

Figure 10: skillFactory package UML

- **skill** : abilities divided into attack and heal, further specified on a target (`Single`, `Multiple`) or behaviour (`Poisonous`) basis via the decorator design pattern. The structure of the decorator pattern allows the different kinds of skills to be stacked without having to create a different class for every possible combination.

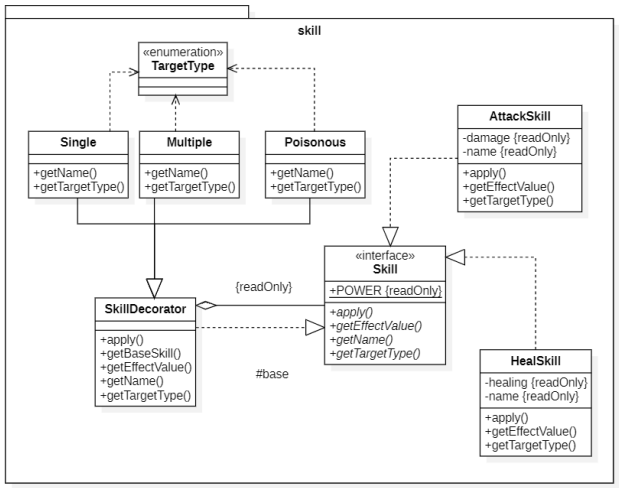

Figure 11: skill package UM

- **character factory** : factory method design pattern generating the characters, to decouple creation from usage and to better define the creation process, during a potential actual development of the game.

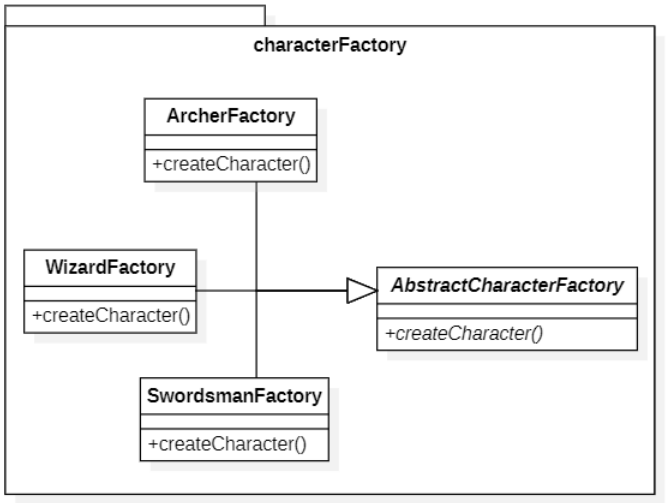

Figure 12: characterFactory package UML

- **character** : playable characters.

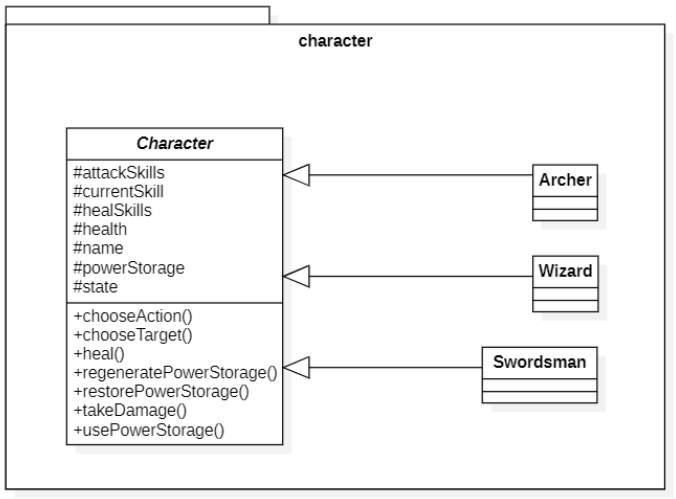

Figure 13: character package UML

- **state** : possible states characters can find themselves in at each turn, consequently to the skills they get subjected to. At any given time, characters can and must be in one and only one state.

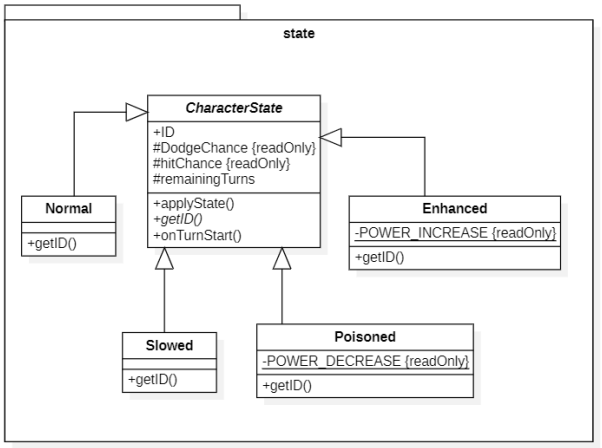

Figure 14: state package UML

3 Implementation

This section focuses on the implementation of the game engine, showing the architecture implemented as a Java program.

The objective of the employment of the pattern described below is that of observing the fundamental principles of Object-Oriented programming.

The intent was to create a robust system, with high cohesion within the same module and low coupling between different components.

The game domain was broken up into hierarchies of classes with specific responsibilities to guarantee easy to test code and a high flexibility: adding new functionality should be possible through extension only, without the need to touch existing code.

The following sections contain a descriptive analysis of all design patterns used throughout the project. This analysis is made in terms of problems solved due to the employment of a certain pattern, and gains achieved.

3.1 `observer`, `engine`

3.1.1 Pattern Observer

The `BattleEngine` should be able to communicate any changes and events that occurred during a turn to external systems, such as a logging system or a future graphic interface. In order to decouple first from the others, this responsibility is entrusted to the Observer pattern: in the `observer package` one can find the interface `Observer` and the abstract class `Observable`, the first being implemented by `Logger`, the latter being a generalisation of `BattleEngine`.

This way, `BattleEngine` is able to communicate through the protected method `NotifyObservers(Object)`, which calls `update(Object)` for each of the, potentially infinite, observers.

The logger can now translate into human readable messages, anything happening during the battle: passed as instances of the `Event` class. This class was defined as a `Record` of all possible events, a data carrier whose explicit immutability guarantees that the information is protected from unexpected changes.

Any specific detail is fed to this class as `TurnInfo`, which acts as a payload, translatable into any kind of information needed for the particular event it concerns. The purpose of the latter is to be a compact container of all relevant information concerning a single action or turn: who acted (`Character player`), what they did (`Command command`), whether it succeeded (`boolean success`), who was affected (`Character target`), and the value of the effect (`int value`) in terms of damage or healing.

3.1.2 Pattern Singleton

`BattleEngine` being the only core coordinator of the battle, its instantiation is controlled through the Singleton pattern, which guarantees a single global access point to the state of the current battle.

3.2 command

3.2.1 Pattern Command

`Command` is used to encapsulate the more complex calculi involved in characters' actions, through the pattern Command.

This abstract class acts as a bridge between `character` and `engine` packages, relieving `Character` of this further responsibility.

`AttackCommand` and `HealCommand` extend the base class, each implementing the abstract method `execute(BattleEngine)` defined in `Command`, in a specific way with regard to the two possible actions a character can do: using an `AttackSkill` or a `HealSkill`.

The method signature decouples this class from the invoker, making it unaware and independent of the `BattleEngine` instance, while the return type being a `List<Event>` keeps it so that the job to manage the events remains the engine's responsibility.

3.3 skillFactory, characterFactory

3.3.1 Pattern Factory Method

The Factory Method pattern is used to encapsulate the complexity of the initialization process for both skills and characters and, above all, to ensure decoupling between creation and usage.

Hence, the instantiation logic has been delegated to a Factory hierarchy: the abstract factories declare the abstract methods for object creation, but defer to the concrete subclasses the decision of which specific class to instantiate.

3.4 skill

3.4.1 Pattern Decorator

The base skills, `AttackSkill` and `HealSkill` which differentiate on the effect they cause on the target, are subjected to two kind of alterations, one based on target type and one that is behavioural.

The various combinations of alterations applicable to the skills are managed through the Decorator pattern to avoid what would quickly become an explosion of classes. The process of creating a new configuration consists into the wrapping of a base skill with decorations, which alter the skill's behaviour by, for example, extending the `apply()` method with additional or modified effects on the target.

`SkillDecorator` is the abstract base class for the decorations, the set of which can be enriched by simply adding a new class inheriting from this base class, allowing maximal flexibility.

Another advantage of this approach is the possibility for the client to treat the decorated skills as simple base skills, with the help of the attribute `base` maintained into the decorator, referencing the `Skill` type object at the base of the decorations.

3.5 character, state

The `Character` class acts as the abstract base class defining the general behaviour for its extensions, which differentiate primarily in their specific skill

sets. Since all characters share core mechanics like power usage, damage infliction, and healing, this abstraction avoids code duplication and prevents the conceptual error of instantiating a generic character. Furthermore, this structure provides a foundation for a potential GUI implementation.

3.5.1 Pattern State

During a battle, characters are subjected to various skills that dynamically alter their performance (e.g., hit or dodge chances). To free the `Character` class from the responsibility of managing these complex conditional behaviours, the State pattern has been employed, ensuring the Single Responsibility Principle is observed.

`CharacterState` acts as the abstract base class for the various states, with concrete subclasses encapsulating the specific rules for each condition. This layout strictly adheres to the Open/Closed Principle, making it easy to introduce new states or modify existing ones without altering the `Character` class.

4 Test

The JUnit 4 framework was employed to execute unit tests of the classes.
The documented tests are the most relevant to the Use Case Templates (Section 2.2). In the repository mentioned in Section 1.2, one can find all the tests executed which cover the core engine logic. This section concentrates on the surface level of the user experience.

The following is the key to the code assigned to every Test which the Use Case Templates refer to.

TST#1 -*AttackCommandTest.executeTest()*
TST#2 - *AttackCommandTest.dodgeTest()*
TST#3 - *AttackSKillTest.skillAppliesSlowedStateTest()*
TST#4 - *HealCommandTest.executeTest()*
TST#5 - *HealCommandTest.enhancedStateTransitionTest()*
TST#6 - *MultipleTest.applyToMultipleTargetsTest()*
TST#7 - *PoisonousTest.skillAppliesStateTest()*

4.1 AttackCommandTest

This test class focuses on verifying the core logic of offensive actions. It includes cases to ensure that damage is correctly calculated and subtracted from a target's health points during an ordinary execution, as well as to verify that health remains unchanged if a target successfully dodges the attack.
Prior to each test execution, the setup phase instantiates both the player and the target, and initializes a new instance of the BattleEngine.

```
1   @Test
2       public void executeTest() {
3               player.setState(new Normal() {
4                       @Override
5                       public float getHitChance() {
6                               return 1.0f;
7                       }
8               });
9               target.setState(new Normal() {
10                      @Override
11                      public float getDodgeChance() {
12                              return 0.0f;
13                      }
14              });
15              Command command = new AttackCommand(player.
                    getAttackSkills().get(0));
16              command.setPlayer(player);
17              command.setTarget(target);
18              int initialHealth = target.getHealth();
19              command.execute(engine);
20
21              assertTrue(target.getHealth() < initialHealth);
22      }
```

Listing 1: AttackCommandTest.exectueTest() - TST#1

```
1       @Test
2       public void dodgeTest() {
3               player.setState(new Normal() {
4                       @Override
5                       public float getHitChance() {
6                       return 0.0f;
7                       }
8               });
9               target.setState(new Normal() {
10                      @Override
11                      public float getDodgeChance() {
12                              return 1.0f;
13                      }
14              });
15              Command command = new AttackCommand(player.
                    getAttackSkills().get(0));
16              command.setPlayer(player);
17              command.setTarget(target);
18              int initialHealth = target.getHealth();
19              command.execute(engine);
20
21              assertEquals(initialHealth, target.getHealth());
22      }
```

Listing 2: AttackCommandTest.dodgeTest() - TST#2

4.2 AttackSkillTest

This test class is designed to verify the correct application and extraction of the base skills from their decorators. To isolate the interaction from the engine's management, the setup phase creates an Archer as the player and a Wizard as the target; additionally, it retrieves the first attack skill from the player's skill set and extracts its base skill component.
The test ensures that the underlying mechanics, such as the correct base damage application and the interaction between a skill and the target's health, function as expected upon a successful hit.

```
1       @Test
2       public void skillAppliesSlowedStateTest() {
3               target.takeDamage(50);
4
5               skill.apply(player, target);
6
7               assertEquals(CharacterState.ID.SLOWED, target.getState
                    ().getID());
8       }
```

Listing 3: AttackSkillTest.skillAppliesSlowedStateTest() - TST#3

4.3 HealCommandTest

The purpose of this class is to validate the restorative mechanics of the engine. The test cases check that healing skills correctly increase a teammate's health up to the maximum threshold and verify the state transition logic, specifically ensuring that a character enters the "Enhanced" state if their health reaches or exceeds a value of 90 after being healed.
During the setup phase, a Wizard is instantiated as the player alongside a fresh instance of the BattleEngine.

```
1       @Test
2       public void executeTest() {
3               player.takeDamage(60);
4               int damagedHealth = player.getHealth();
5
6               Skill rawSkill = player.getHealSkills().get(0);
7               HealSkill baseHealSkill = (HealSkill) ((SkillDecorator)
                     rawSkill).getBaseSkill();
8
9               Command command = new HealCommand(player.getHealSkills
                    ().get(0));
10              command.setPlayer(player);
11              command.setTarget(player);
12
13              command.execute(engine);
14
15              assertEquals(player.getHealth(), damagedHealth+
                    baseHealSkill.getEffectValue());
16      }
```

Listing 4: HealCommandTest.executeTest() - TST#4

```
1       @Test
2       public void enhancedStateTransitionTest() {
3               assertEquals(CharacterState.ID.NORMAL, player.getState
                    ().getID());
4
5               player.takeDamage(20);
6
7               Command command = new HealCommand(player.getHealSkills
                    ().get(0));
8               command.setPlayer(player);
9               command.setTarget(player);
10
11              command.execute(engine);
12
13              assertTrue(player.getHealth() >= 90);
14
15              assertEquals(CharacterState.ID.ENHANCED, player.
                    getState().getID());
16      }
```

Listing 5: HealCommandTest.enhancedStateTransitionTest() - TST#5

4.4 MultipleTest

This class tests the functionality of the decorator pattern applied to skills that affect multiple targets. It ensures that when a "Multiple" decorated skill is executed, damage is simultaneously and correctly applied to all selected targets in the opposing team, confirming the decorator's ability to handle multiple character references.
To prepare the testing environment, the setup initializes a Wizard as the player and two distinct targets (an Archer and a Swordsman) to simulate a multi-target scenario.

```
1        @Test
2        public void applyToMultipleTargetsTest() {
3                Skill skill = new Multiple(new AttackSkill("Fireball"))
                     ;
4
5                List<Character> targets = Arrays.asList(target1,
                     target2);
6
7                int t1InitialHealth = target1.getHealth();
8                int t2InitialHealth = target2.getHealth();
9
10               skill.apply(player, targets);
11
12               assertTrue(target1.getHealth() < t1InitialHealth);
13               assertTrue(target2.getHealth() < t2InitialHealth);
14       }
```

Listing 6: MultipleTest.applyToMultipleTargetsTest() - TST#6

4.5 PoisonousTest

This test verifies the behavioural decoration of skills that inflict status alterations. It specifically checks that a ”Poisonous” decorated attack not only inflicts damage, but also successfully triggers a state transition in the target, changing their status from ”Normal” to ”Poisoned” upon a successful hit. The setup initializes a basic combat scenario by creating an Archer to act as the player and a Wizard as the target.

```
1         @Test
2         public void skillAppliesStateTest() {
3                 Skill skill = new Single(new Poisonous(new AttackSkill(
                      "arrow")));
4                 assertEquals(CharacterState.ID.NORMAL, target.getState
                      ().getID());
5
6                 skill.apply(player, target);
7
8                 assertEquals(CharacterState.ID.POISONED, target.
                      getState().getID());
9         }
```

Listing 7: PoisonousTest.skillAppliesStateTest() - TST#7